通知抽取助手 · 技术文档

> 队伍：Triple Y ｜ 比赛：传智杯 AI WEB 网页开发挑战赛 版本：v1.0 ｜ 日期：2026-10-26
>
> 说明：本文由 C 汇总。第 4.1 / 4.2 节（模型选型、Prompt 设计）由 A 补充，其余章节已完成初稿。

目录

1. 项目概述
2. 需求分析
3. 系统架构设计
4. AI 技术方案（4.1 模型选型 / 4.2 Prompt 设计 / 4.3 输出稳定性 / 4.4 多模态与 OCR / 4.5 RAG 与可解释性）
5. 核心功能实现
6. 测试与部署
7. 创新点
8. 总结与展望 附录：运行命令、环境变量与测试数据

---

1. 项目概述

问题背景：校园与职场中的"办事通知"以纯文字、截图、PDF 等形式散落在群消息和公告里，关键信息（截止时间、材料清单、办理步骤、审核要求）分散在长文中。人工阅读容易漏材料、看错时间、混淆流程；通用大模型直接回答又容易编造，用户无法核验。

一句话价值：把一段通知变成**可勾选、可溯源、可提醒**的待办清单。

目标用户：在校学生（奖助学金、竞赛报名、证件办理）、社团/单位内勤（活动立项、经费报销、场地申请）、职场人士（入职材料、资质办理、行政审批）。

本期范围（P1）：文字通知解析、截图识别、文字型 PDF 提取、三档结构化输出、材料清单与缺项核对、查看依据、材料库检索与追问、清单保存与恢复。 **后续规划（P2）**：多文件合并解析、复杂条件分支、日历同步、多人协作。

2. 需求分析

2.1 典型场景

- 学生收到一条奖学金通知：需要知道**截止时间**、要交**哪些材料**、**交给谁/交到哪**、**先做什么后做什么。**
- 社团收到立项通知：需要知道**审核环节**、**依赖关系**（审核通过后才能缴费）、**风险点**（时间紧、材料复用）。
- 用户手上已有部分材料：需要系统告诉他**还缺什么**，而不是重复罗列全部要求。

2.2 功能需求

编号	需求	验收标准
F1	文字通知 → Matter JSON	返回 title / deadline / materials / step_list 等，结构符合 Schema
F2	三档复杂度自适应	simple 不出现材料/步骤；standard 有材料与步骤；complex 增加审核/依赖/预警
F3	用户情况核对	勾选材料 + 自述情况 → 已准备 / 缺少 / 需要确认 三类
F4	查看依据	每条要求可查看原文片段；无依据标"系统建议"
F5	截图解析	上传截图 → 识别文字 → Matter；格式/大小受限
F6	PDF 解析	文字型 PDF → Matter；扫描版明确提示
F7	材料库与追问	多材料上传、片段编号、检索、回答附出处、材料外拒答
F8	保存与恢复	准备清单/修改内容保存后，刷新或重启仍可恢复

2.3 非功能需求

维度	要求	实现
稳定性	模型不可用时服务不崩、前端有可读结果	视觉→本地 OCR→Mock 三层降级；统一错误信封
可解释	关键结论能追溯到原文	`source_ref.quote` + `sources` + 片段编号 + `is_suggestion`
性能	联调/演示响应可接受	Mock 实测 100 次：平均 10.4ms、P95 16.9ms；真实模型 10~30s（已在前端设 90s 超时）
安全性	密钥不入库、材料外不乱答、输出不注入	`.env` + `.gitignore`；无依据拒答；前端 `esc()` 转义
可维护性	结构清晰、可自测	8 套自测脚本 + `preflight.py` 一键自检 + `check_sync.py`

---

3. 系统架构设计

系统采用前后端分离的三层结构：

```
浏览器（前端 5173 / 静态托管）
        |  HTTP + JSON / multipart
        ▼
FastAPI 后端（8000）
  ├── 入口层：CORS、参数校验、统一响应、异常兜底
  ├── 能力层：文本抽取 / 图片解析 / PDF 提取 / 准备清单 / 查看依据 / 文件检索 / 追问
  ├── 抽取层：A 的 service.py（Prompt + LLM + Matter 结构 + Schema 校验）
  └── 存储层：backend/data（准备清单、文件片段索引，JSON 原子写）
        |
        ▼
大模型（OpenAI 兼容接口，可配置）
  文本模型：LLM_MODEL        视觉模型：TEAM_LLM_MODEL
  无 Key 时：离线 Mock + 本地 OCR（RapidOCR）
```

核心设计原则：**模型只负责"读懂"，结构、判定与降级由代码兜底**，保证演示可复现、失败有反馈。

接口清单

接口	作用
`POST /api/analyze`	文字 → Matter
`POST /api/check`	用户勾选 + 自述 → 缺项核对
`POST /api/prepare-list` / `save` / `GET`	准备清单生成 / 保存 / 读取
`POST /api/evidence`、`/index`	要求 → 原文依据（无依据标为系统建议）
`POST /api/analyze-image`	截图 → Matter（视觉模型 / 本地 OCR）
`POST /api/analyze-pdf`	文字型 PDF → Matter
`POST /api/files`、`GET /api/files`、`POST /api/files/search`	材料上传、片段编号、检索
`POST /api/ask`	先检索再回答，附出处，材料外拒答
`GET /health`	服务状态与当前模式（mock/real）

所有接口统一返回 `{success, data, error}`，且始终 HTTP 200，前端只需判断 `success`。

4.1 模型选型（A 补充）

> TODO（A）：说明文本模型与视觉模型的选型理由（中文能力、JSON 输出、成本、OpenAI 兼容性）、以及团队 Key 的配置方式（`LLM_*` / `TEAM_LLM_*`）与降级策略。

4.2 Prompt 设计（A 补充）

> TODO（A）：从 `src/prompt.py` 提炼： 1. 角色与任务边界（只抽原文、禁止编造） 2. 三档模式的判定规则与 `enabled_sections` 3. 字段互斥规则（simple 禁止材料/步骤等） 4. 步骤/材料命名与依赖格式（`step_1` / `mat_1` / `depends_on`） 5. 三模式输出示例（few-shot）如何稳定结构

4.3 输出稳定性（C 侧实现）

1. **复杂度重算**：不采信模型给的分数，用固定公式 `0.2 • deadline + 0.1 • 多提醒 + 0.2 • 多步骤 + 0.2 • 材料 + 0.1 • 审核 + 0.1 • 依赖 + 0.1 • 周期 + 0.1 • 子步骤`， 再按 `has_sub_steps/has_review/has_dependencies → complex`、`<0.3 simple`、`0.3 – 0.6 standard`、`>0.6 complex` 判定。
2. **字段裁剪**：`enforce_mode` 按模式删除互斥字段（simple 不出现 materials/step_list/todo_list 等），保证前端渲染契约稳定。
3. **Schema 校验**：`jsonschema` 校验 Matter 结构，失败直接返回可读错误，不把脏数据交给前端。
4. **三层降级**：真实模型 → 本地 OCR（图片）→ 离线 Mock（无 Key），任一层可用即可演示。

4.4 多模态与 OCR（C 侧实现）

- **截图**：`POST /api/analyze-image`，multipart 字段 `image`，JPG/PNG/WEBP，≤5MB。 有团队 Key 时调用 `analyze_notification_from_image`（多模态 + 截图专用规则：表格/人数/日期/材料/金额，模糊写入 `confirmation_needed`）； 无 Key 或视觉调用失败时，自动用 **RapidOCR（onnxruntime）** 本地识别，再走统一抽取管道。
- **PDF**：`POST /api/analyze-pdf`，字段 `file`，≤10MB、≤30页，仅文字型；用 pypdf 提取文字。 扫描版提取不到文字时返回"请改用清晰截图或直接粘贴文字"，避免空结果。
- 统一限制端点：`/api/analyze-image/limits`、`/api/analyze-pdf/limits`，前端提示与后端完全一致。

4.5 RAG 与可解释性（C 侧实现）

- **片段编号**：上传材料按 400 字 / 80 字重叠切分，生成 `file_xxx_c1`、`_c2`…，记录 `char_start/char_end`。
- **检索**：`POST /api/files/search` 对片段做中文二元组重叠打分（命中片段 + 精确子串加权），返回文件名、片段号、原文、分数。
- **追问**：`POST /api/ask` 先检索；有 Key 时把检索片段作为上下文交给 `ask_question`，无 Key 时直接返回最相关原文与出处。
- **不敢答**：检索为空时返回"在已上传的材料中没有找到…无法回答"，不调用模型编造。
- **依据**：`POST /api/evidence` 对材料/步骤/待办/附件/截止/预警/要点返回 `source_ref.quote`； 无引用的条目标 `is_suggestion=true`，前端以黄色"系统建议"展示。
- **双通知对比**：`POST /api/compare` 分别解析并保留两份通知的文件名、原文和 Matter，对比截止时间、材料与步骤差异；记录支持按会话读取和删除。

5. 核心功能实现（后端）

- **统一入口**：`main.py` 负责 CORS、参数校验（空文本 / 超长 10000 字）、multipart 大小与格式校验、统一异常兜底。
- **统一管道**：`analysis_pipeline.analyze_with_fallback` 负责"真实模型 / 离线 Mock"切换，文本、PDF、图片三条入口共用。
- **存储**：准备清单（`data/prepare_lists.json`）与文件索引（`data/files.json`）采用线程锁 + 临时文件原子写，容器通过 volume 持久化。
- **状态联动**：勾选材料 → 关联待办自动 `completed`；依赖全部完成 → `blocked` 解除为 `pending`；状态枚举兼容 `pending/completed/blocked/need_confirm`。
- **版本与导出**：准备清单每次确认保存后 `version` 递增，保留 `previous_todo_list` 和最近 5 版 `history`；支持服务端导出 CSV（UTF-8 BOM）与 Markdown。
- **日期核对**：`GET /api/tools/check-date` 返回星期、距今、周末、常见法定节假日与“是否工作日”，并提示调休需人工确认。
- **会话隔离与删除**：前端通过 `X-Session-Id` 区分浏览器会话，文件库、准备清单、通知对比互不可见；三类数据均提供删除接口。
- **限额与密钥安全**：真实模型模式默认按 IP + 路径限流（普通接口 60 次/分钟、上传 10 次/分钟），Key 只从环境变量读取，不进入仓库。

6. 测试与部署

自动化测试（9 套，全部通过）

脚本	项数	覆盖
`smoke_test.py`	10	健康检查、正常/空/超长文本、非法 JSON、CORS、Mock 性能
`smoke_test_check.py`	10	缺项差集、全勾选、自述冲突、参数兜底
`smoke_test_prepare.py`	11	生成、保存、读取、修改内容持久化
`smoke_test_evidence.py`	8	索引、单条依据、无依据 → 建议、参数校验
`smoke_test_image.py`	6	limits、合法图、格式/大小/空文件
`smoke_test_pdf.py`	6	文字型 PDF、扫描件、页数/大小限制
`smoke_test_files.py`	10	上传、片段编号、检索、追问、材料外拒答
`smoke_test_acceptance.py`	3 样本	三档模式端到端验收
`smoke_test_hardening.py`	16	双通知、版本历史、导出、日期、会话隔离、删除
`preflight.py`	13	演示前一键自检（含 10/28–10/30 新链路）

部署

- Docker：`backend/Dockerfile`（含 pypdf、RapidOCR）+ `docker-compose.yml`，`backend/data` volume 持久化，内置 HEALTHCHECK。
- Nginx 反代 + HTTPS，`proxy_read_timeout 180s`、`client_max_body_size 12m`。
- 详见 `docs/部署说明.md`。

---

7. 创新点

7.1 技术创新

1. **公式化复杂度判定 + 字段裁剪**：不让模型"凭感觉"给分，服务端用固定公式重算 complexity_score，并按模式强制裁剪字段，保证同一份通知的输出稳定、可复现。
2. **三档结构自适应**：simple / standard / complex 按需输出，前端只渲染 `enabled_sections` 中声明的区块，避免"简单通知给一堆空模块"。

3. **全链路降级**：视觉大模型不可用 → 本地 OCR（RapidOCR）→ 离线 Mock；文本模型不可用 → 离线抽取。演示与前端联调不会因为 Key/网络中断而白屏。
4. **可解释与防幻觉**：字段级 `source_ref` + "查看依据"接口；无原文依据的条目标为"系统建议"；材料库检索不到时直接回答"未找到依据"，不编造。

7.2 应用创新

1. **从"读通知"到"能执行"**：输出不是一段摘要，而是可勾选的材料清单、可排序的步骤、可提醒的 DDL。
2. **用户情况驱动的缺项核对**：把"通知要求"和"用户已有"做差集，直接告诉用户还缺什么、什么需要确认。
3. **材料库 + 追问**：上传多份材料后可以问"视频有什么要求"，答案附文件名与片段编号，形成"办事资料助手"。

8. 总结与展望

已完成：文字/截图/PDF 三种输入通道；三档结构化输出；材料清单与缺项核对；查看依据；材料库片段检索与追问；准备清单保存与恢复；8 套自动化测试与一键自检；Docker 部署方案与演示素材。

局限：扫描版 PDF 依赖截图通道；复杂表格/手写体的识别精度取决于模型；无团队 Key 时为演示数据（结构真实、内容虚构）。

展望：多文件合并解析、复杂条件分支（"若为应届生需额外提交…"）、日历/飞书同步、多人协作清单。

附录

A.1 运行命令

```
cd backend
.\.venv\Scripts\python.exe -m uvicorn main:app --reload --host 0.0.0.0 --port 8000
cd ..\frontend
npm run dev

# 重新生成技术文档 HTML + PDF（A 修改 4.1/4.2 后运行）
powershell -ExecutionPolicy Bypass -File ..\scripts\build_tech_pdf.ps1
```

A.2 环境变量

变量	说明
`LLM_ENABLE_MOCK`	无 Key 时是否走离线 Mock
`LLM_API_KEY` / `LLM_BASE_URL` / `LLM_MODEL`	文本模型
`TEAM_LLM_API_KEY` / `TEAM_LLM_BASE_URL` / `TEAM_LLM_MODEL`	视觉模型（截图）
`MAX_TEXT_LENGTH`	文本长度上限（默认 3000，演示配 10000）
`RATE_LIMIT_PER_MINUTE`	普通模型接口每分钟限额（默认 60）
`UPLOAD_RATE_LIMIT_PER_MINUTE`	上传接口每分钟限额（默认 10）
`RATE_LIMIT_ENFORCE_IN_MOCK`	Mock 模式是否强制限流（默认 0）
`X-Session-Id`	前端请求头，用于用户数据隔离（不是登录鉴权）

A.3 测试结果

脚本	结果
8 套自测（接口/缺项/清单/依据/图片/PDF/文件/追问）	全部通过
`smoke_test_acceptance.py`	simple / standard / complex 3/3
`preflight.py`	13/13，可开始演示
`smoke_test_hardening.py`	16/16
`smoke_test_perf.py`（Mock 100 次）	平均约 10–14ms，P95 约 17ms
`docs/技术文档.pdf`	已生成 9 页草稿（≤30 页）；A 补完 4.1/4.2 后需重导